JavaScript Foundations

From Syntax to the DOM

Seyed Ahmad Mousavi Zadeh

Variables, data, functions, objects, DOM, async, and APIs

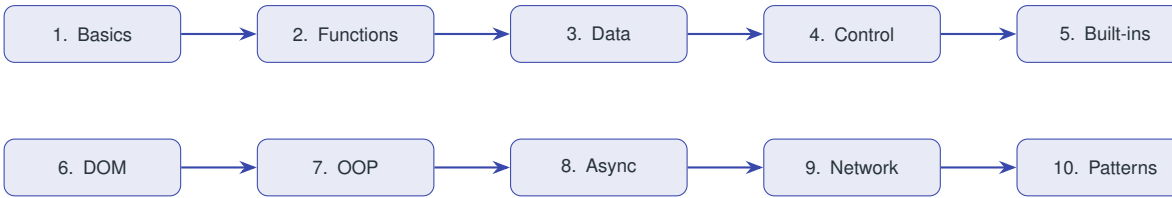
A complete, example-driven course

Learning outcomes

By the end of this complete course, learners will be able to:

- Write JavaScript inline, internally, and externally.
- Use variables, all data types, operators, and strict equality.
- Write functions, arrow functions, closures, and control `this`.
- Master arrays, strings, objects, and their built-in methods.
- Control flow with `if/else`, `switch`, `for`, `while`, `do-while`, `map`, `forEach`, `filter`, `reduce`.
- Use `Math`, `Date`, `JSON`, `Intl`, and regular expressions.
- Manipulate the DOM, handle events, and control CSS from JavaScript.
- Apply OOP: classes, inheritance, private fields, getters/setters.
- Work with Promises, `async/await`, `fetch`, Axios, and the Event Loop.
- Persist data with `localStorage`, `sessionStorage`, and cookies.
- Organize code with ES Modules and avoid common async pitfalls.

Course roadmap



How to use this deck

Each part builds on the previous one. Run every code example in the browser console as you go.

Part 1 — Basics

A short history of JavaScript



- Created in **10 days** in 1995 at Netscape.
- Standardized as **ECMAScript**; JavaScript is its most popular implementation.
- ES5 (2009) modernized the language; ES6 (2015) added `let`/`const`, arrow functions, classes, promises, modules.
- Modern JS runs in browsers, on servers (Node.js), in mobile apps, and on edge devices.

Three ways to write JavaScript

Inline

```
<button onclick="alert('Hi')">
  Click
</button>
```

Attached to one element. Hard to maintain. Avoid for large apps.

Internal

```
<script>
  console.log("Hello");
</script>
```

Inside the HTML file. Fine for demos and small pages.

External

```
<script src="app.js"
        defer></script>
```

Separate file. Recommended for real projects.

Best practice

Use external scripts, place them at the end of `body` or use `defer`/`async`, and keep markup and behavior separate.

defer vs async

`defer`

- Downloads in parallel.
- Executes after HTML parsing.
- Preserves script order.
- Best for most apps.

`async`

- Downloads in parallel.
- Executes as soon as ready.
- Order not guaranteed.
- Best for independent scripts (analytics).

Without either attribute, a script blocks HTML parsing until it finishes downloading and executing.

Your first tool: the console

```
console.log("Hello, world!");
console.warn("Be careful");
console.error("Something broke");
console.table([{ a: 1 }, { a: 2 }]);
console.time("loop");
for (let i = 0; i < 1000; i++) {}
console.timeEnd("loop");
```

`console.log` is the fastest way to inspect values while learning.

Browser DevTools

Open with `F12` or `Ctrl+Shift+I`. Use the **Console** tab to run JavaScript live and inspect values.

Variables: var, let, const

```
var oldStyle = "function-scoped";
let count = 0;       // reassignable
const PI = 3.14159; // cannot reassign
count = count + 1;
// PI = 3; // TypeError
```

Rules of thumb

- Prefer `const` by default.
- Use `let` when reassignment is needed.
- Avoid `var` in modern code.

`var` is function-scoped and hoisted differently. `let`/`const` are block-scoped and safer.

JavaScript data types

Primitives

- `string` — text
- `number` — integers and floats
- `boolean` — true/false
- `null` — intentional absence
- `undefined` — not yet assigned
- `symbol` — unique identifier
- `bigint` — very large integers

Reference types

- `object` — key/value pairs
- `array` — ordered list (an object)
- `function` — callable object
- `Date`, `Map`, `Set`, etc.

Use `typeof` to inspect a type. `typeof null` returns `"object"` (a historical quirk).

Examples of each data type

```
const name    = "Ada";          // string
const age     = 36;             // number
const pi      = 3.14;           // number (float)
const isAdmin = true;           // boolean
const nothing = null;           // null
let   pending;                  // undefined
const id      = Symbol("id");   // symbol
const big     = 9007199254740993n; // bigint

const user    = { name: "Ada", age: 36 }; // object
const scores  = [90, 85, 77];             // array
function greet() { return "Hi"; }         // function
```

Strings use `""`, `''`, or backticks. Backticks allow interpolation: `` `Hello, ${name}` ``.

Type coercion and equality

```
"5" == 5     // true  (loose)
"5" === 5    // false (strict)
0 == false   // true
0 === false  // false
null == undefined  // true
null === undefined // false
Number("42")   // 42
String(42)     // "42"
Boolean("")    // false
```

Rule

Always prefer === and !==. They compare both value and type, which prevents surprising bugs.

Operators overview

Arithmetic

`+ - * / % **`

`++` and `--` increment/decrement

Comparison

`== === != !== > < >= <=`

Logical

`&&` (and), `||` (or), `!` (not), `??` (nullish)

Operator precedence matters. Use parentheses to make intent clear.

Assignment

`= += -= *= /= %=`

String

`+` concatenation, template literals

Ternary

```
condition ? a : b
```

Operator examples

```
let a = 10, b = 3;

a + b         // 13
a - b         // 7
a * b         // 30
a / b         // 3.333...
a % b         // 1   (remainder)
a ** b        // 1000 (exponent)

a > b         // true
a === 10      // true
a !== b       // true

(a > 5) && (b < 5)   // true
(a > 20) || (b < 5)  // true
!true                // false

const label = a > 5 ? "big" : "small"; // "big"
```

Template literals

```
const name = "Ada";
const age  = 36;

// Old way
const msg1 = "Hello, " + name + "! You are " + age + ".";

// Template literal
const msg2 = `Hello, ${name}! You are ${age}.`;

// Multi-line
const html = `
  <div class="card">
    <h2>${name}</h2>
  </div>
`;

// Expression inside
const total = `Total: ${(10 + 5) * 2}`; // "Total: 30"
```

Template literals use backticks `` `...` ``, support `${expression}` interpolation and multi-line strings.

Part 2 — Functions

Function declarations and expressions

```
// Declaration (hoisted)
function add(a, b) {
  return a + b;
}

// Expression (not hoisted)
const sub = function (a, b) {
  return a - b;
};

// Arrow function
const mul = (a, b) => a * b;

// Default parameters
function greet(name = "friend") {
  return `Hello, ${name}!`;
}
```

Return values

A function without `return` returns `undefined`. Arrow functions with a single expression return it implicitly.

Arrow functions in detail

```
// Single parameter, no parentheses needed
const square = n => n * n;

// Multiple parameters
const add = (a, b) => a + b;

// Multiple statements
const log = (msg) => {
  console.log(msg);
  return msg.length;
};

// Returning an object literal needs parentheses
const makeUser = (name) => ({ name });

// No own this, arguments, or prototype
```

Arrow functions inherit `this` from the enclosing scope. They cannot be used as constructors.

Parameters: rest, spread, destructuring

```
// Rest parameters
function sum(...nums) {
  return nums.reduce((a, b) => a + b, 0);
}
sum(1, 2, 3); // 6

// Spread into a call
const values = [4, 5, 6];
sum(...values); // 15

// Destructuring parameters
function show({ name, age }) {
  return `${name} is ${age}`;
}
show({ name: "Ada", age: 36 });
```

Function methods: call, apply, bind

```
function introduce(greeting) {
  return `${greeting}, I am ${this.name}`;
}

const user = { name: "Ada" };

introduce.call(user, "Hi");     // "Hi, I am Ada"
introduce.apply(user, ["Hey"]); // "Hey, I am Ada"

const bound = introduce.bind(user);
bound("Hello");                 // "Hello, I am Ada"
```

- `call` and `apply` invoke immediately with a chosen `this`.
- `bind` returns a new function with `this` fixed.
- Arrow functions do not have their own `this`.

Understanding this

```
const user = {
  name: "Ada",
  regular() {
    return this.name; // "Ada"
  },
  arrow: () => this.name, // undefined (or outer this)
};

// Detached method loses this
const fn = user.regular;
fn(); // TypeError or undefined

// Fix with bind
const safe = user.regular.bind(user);
safe(); // "Ada"
```

`this` depends on *how* a function is called, not where it is defined (except arrow functions).

Closures

```
function makeCounter() {
  let count = 0;
  return function () {
    count++;
    return count;
  };
}

const next = makeCounter();
next(); // 1
next(); // 2
next(); // 3
```

Why it matters

A closure lets an inner function remember variables from its outer scope even after the outer function has returned. It is the basis for modules, event handlers, and private state.

Immediately Invoked Function Expressions (IIFE)

```
(function () {
  const secret = "private";
  console.log("Runs immediately");
})();

// Arrow IIFE
(() => {
  console.log("Also runs immediately");
})();
```

IIFEs were historically used to create private scopes before `let`/`const` and modules. They are still useful for encapsulation.

Part 3 — Arrays, Strings, Objects

Arrays: creating and indexing

```
const fruits = ["apple", "banana", "cherry"];
fruits[0];          // "apple"
fruits.length;      // 3
fruits[fruits.length - 1]; // "cherry"

const mixed = [1, "two", true, null];
const empty = [];
const grid = [[1,2],[3,4]]; // 2D array
```

Remember

Arrays are zero-indexed. Accessing an out-of-range index returns `undefined`, not an error.

Mutating array methods: push, pop, shift, unshift

```
const stack = [1, 2, 3];

stack.push(4);     // [1,2,3,4]  add to end
stack.pop();       // [1,2,3]    remove from end
stack.unshift(0);  // [0,1,2,3]  add to start
stack.shift();     // [1,2,3]    remove from start

// splice: remove/insert anywhere (mutates)
const arr = ["a","b","c","d"];
arr.splice(1, 2);        // removes "b","c" -> ["a","d"]
arr.splice(1, 0, "x");   // inserts "x" at index 1
```

`push`/`pop` work at the end; `unshift`/`shift` work at the start. `splice` mutates the original array.

Non-mutating array methods

```
const nums = [3, 1, 4, 1, 5];

nums.slice(1, 3);         // [1, 4]   (copy range)
nums.concat([9, 2]);      // [3,1,4,1,5,9,2]
nums.indexOf(4);          // 2
nums.lastIndexOf(1);      // 3
nums.includes(5);         // true
nums.join("-");           // "3-1-4-1-5"

const sorted = [...nums].sort((a, b) => a - b);
// sort mutates; copy first with spread

const reversed = [...nums].reverse();
```

`slice`, `concat`, `indexOf`, `includes`, `join` do not mutate. `sort` and `reverse` do.

Transform arrays: map, filter, reduce

```
const nums = [1, 2, 3, 4, 5];

const doubled = nums.map(n => n * 2);        // [2,4,6,8,10]
const evens   = nums.filter(n => n % 2 === 0); // [2,4]
const total   = nums.reduce((sum, n) => sum + n, 0); // 15

// Chaining
const result = nums
  .filter(n => n > 2)
  .map(n => n * 10)
  .reduce((a, b) => a + b, 0); // 120
```

`map`, `filter`, `reduce` are non-mutating and chainable. Prefer them over manual loops for transformation.

Search and test array methods

```
const users = [
  { name: "Ada", age: 36 },
  { name: "Lin", age: 28 },
  { name: "Sam", age: 42 },
];

users.find(u => u.age > 30);        // { name: "Ada", age: 36 }
users.findIndex(u => u.name === "Lin"); // 1
users.some(u => u.age > 40);        // true
users.every(u => u.age > 18);       // true

const names = users.map(u => u.name);
names.includes("Ada");              // true
```

forEach vs map

`forEach` — side effects

```
const nums = [1, 2, 3];
nums.forEach(n => {
  console.log(n);
});
// returns undefined
```

`map` — transform

```
const doubled = nums.map(n => n * 2);
// [2, 4, 6]
```

Returns a new array. Chainable.

If you need the result, use `map`. If you only need to perform an action, use `forEach`.

Strings and their methods

```
const s = "  Hello, JavaScript!  ";

s.length;                 // includes spaces
s.trim();                 // "Hello, JavaScript!"
s.toUpperCase();
s.toLowerCase();
s.includes("Java");       // true
s.indexOf("Script");      // position
s.slice(2, 7);
s.replace("JavaScript", "JS");
s.replaceAll("l", "L");
"a,b,c".split(",");       // ["a","b","c"]
["a","b","c"].join("-");  // "a-b-c"
"abc".charAt(1);          // "b"
"abc".padStart(5, "*");   // "**abc"
```

Strings are immutable; methods return new strings.

Objects: creation and access

```
const user = {
  name: "Ada",
  age: 36,
  greet() {
    return `Hi, ${this.name}`;
  }
};

user.name;         // "Ada"
user["age"];       // 36
user.greet();      // "Hi, Ada"

user.email = "ada@example.com"; // add
delete user.age;                // remove
"name" in user;                 // true
```

Objects vs arrays

Use objects for named properties and arrays for ordered lists. Arrays are technically objects.

Object methods and utilities

```
const user = { name: "Ada", age: 36 };

Object.keys(user);    // ["name","age"]
Object.values(user);  // ["Ada",36]
Object.entries(user); // [["name","Ada"],["age",36]]

Object.assign({}, user, { city: "London" });
const clone = { ...user };
const merged = { ...user, age: 37 };

const { name, age } = user;          // destructuring
const { name: userName } = user;     // rename

Object.freeze(user); // immutable
Object.seal(user);   // no add/remove, edit ok
```

Spread (`...`) makes shallow copies. For deep copies use `structuredClone(obj)`.

Map and Set

`Map` — key/value

```
const m = new Map();
m.set("a", 1);
m.get("a");         // 1
m.has("a");         // true
m.delete("a");
m.size;
m.forEach((v, k) => console.log(k, v));
```

Keys can be any type.

`Set` — unique values

```
const s = new Set([1,2,2,3]);
s.has(2);           // true
s.add(4);
s.delete(1);
s.size;             // 3
[...s];             // [2,3,4]
```

Use for deduplication.

Part 4 — Control Flow

Conditionals: if, else if, else

```
const score = 82;

if (score >= 90) {
  console.log("A");
} else if (score >= 80) {
  console.log("B");
} else if (score >= 70) {
  console.log("C");
} else {
  console.log("Needs improvement");
}

// Ternary
const grade = score >= 80 ? "Pass" : "Fail";

// Nullish and logical
const name = user.name ?? "Guest";
const label = isAdmin && "Admin";
```

Switch / case

```
const day = "Tue";

switch (day) {
  case "Mon":
  case "Tue":
  case "Wed":
  case "Thu":
  case "Fri":
    console.log("Workday");
    break;
  case "Sat":
  case "Sun":
    console.log("Weekend");
    break;
  default:
    console.log("Unknown day");
}
```

Do not forget `break`; without it, execution falls through to the next case. Use `switch` for many fixed comparisons on one value.

Loops: for, while, do-while

```
for (let i = 0; i < 5; i++) {
  console.log(i);
}
```

```
let i = 0;
while (i < 5) {
  console.log(i);
  i++;
}
```

```
let j = 0;
do {
  console.log(j);
  j++;
} while (j < 5);
```

- `for` — known number of iterations.
- `while` — repeat while a condition holds.
- `do...while` — runs at least once.

for...of and for...in

`for...of` (values)

```
const nums = [10, 20, 30];
for (const n of nums) {
  console.log(n);
}
// 10 20 30
```

`for...in` (keys)

```
const user = { name: "Ada", age: 36 };
for (const key in user) {
  console.log(key, user[key]);
}
// name Ada, age 36
```

Use `for...of` for arrays and iterables. Use `for...in` for object keys, and prefer `Object.keys()` for arrays.

break, continue, labels

```
for (let i = 0; i < 10; i++) {
  if (i === 5) break;       // exit loop
  if (i % 2 === 0) continue; // skip even
  console.log(i);           // 1, 3
}

outer: for (let i = 0; i < 3; i++) {
  for (let j = 0; j < 3; j++) {
    if (j === 1) continue outer;
    console.log(i, j);
  }
}
```

`break` exits the loop; `continue` skips to the next iteration. Labels are rarely needed but useful for nested loops.

Part 5 — Math, Date, JSON, Intl, Regex

The Math object

```
Math.PI;            // 3.14159...
Math.E;             // 2.718...
Math.round(4.6);    // 5
Math.floor(4.9);    // 4
Math.ceil(4.1);     // 5
Math.abs(-7);       // 7
Math.max(1, 5, 3);  // 5
Math.min(1, 5, 3);  // 1
Math.pow(2, 10);    // 1024
Math.sqrt(16);      // 4
Math.random();      // 0 <= x < 1

// Random integer 1..6
const dice = Math.floor(Math.random() * 6) + 1;
```

The Date object

```
const now = new Date();
now.getFullYear();     // 2025
now.getMonth();        // 0-11 (January = 0)
now.getDate();         // 1-31
now.getDay();          // 0-6 (Sunday = 0)
now.getHours();
now.getMinutes();
now.getSeconds();
now.getTime();         // ms since 1970

const iso = now.toISOString();
const local = now.toLocaleString("en-GB");

const past = new Date("2020-01-01");
const diff = now - past; // milliseconds
```

Months are zero-indexed. For serious date math, consider `date-fns` or `Luxon`.

JSON: JavaScript Object Notation

```
const user = { name: "Ada", age: 36 };

// Object -> string
const json = JSON.stringify(user);
// '{"name":"Ada","age":36}'

// String -> object
const obj = JSON.parse(json);
obj.name; // "Ada"

// Pretty print
JSON.stringify(user, null, 2);

// Deep clone
const clone = JSON.parse(JSON.stringify(user));
```

Limits

JSON cannot store functions, `undefined`, or circular references. Use `structuredClone` for modern deep cloning.

Intl: formatting numbers, dates, currencies

```
const n = 1234567.89;

new Intl.NumberFormat("en-US")
  .format(n);           // "1,234,567.89"

new Intl.NumberFormat("de-DE")
  .format(n);           // "1.234.567,89"

new Intl.NumberFormat("en-US", {
  style: "currency",
  currency: "USD",
}).format(n);           // "$1,234,567.89"

new Intl.DateTimeFormat("en-GB", {
  dateStyle: "long",
}).format(new Date());  // "26 October 2026"
```

Use `Intl` instead of manual formatting for locale-aware output.

Regular expressions (basics)

```
const re = /hello/i;        // case-insensitive
re.test("Hello world");     // true

"abc123".match(/\d+/);      // ["123"]
"a,b,c".split(/,/);         // ["a","b","c"]
"Hello".replace(/l/g, "L"); // "HeLLo"

// Common patterns
/^\d{3}-\d{4}$/.test("123-4567"); // phone-like
/^[\w.+-]+@[\w-]+\.[\w.]+$/.test("a@b.com"); // email-like

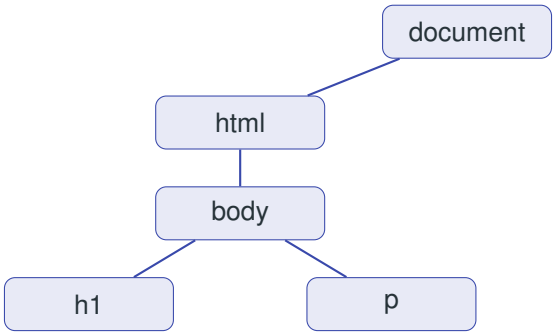
// Named groups
const m = "2025-01-15".match(/(?<y>\d{4})-(?<m>\d{2})/);
m.groups.y; // "2025"
```

Useful flags: `g` (global), `i` (ignore case), `m` (multiline), `u` (unicode).

Part 6 — DOM & Events

What is the DOM?

The Document Object Model is a tree representation of an HTML page that JavaScript can read and modify.



Every element, attribute, and text node is part of this tree. JavaScript changes the page by changing the tree.

Selecting DOM elements

```
// By ID
const title = document.getElementById("title");

// By CSS selector (first match)
const card = document.querySelector(".card");

// All matches
const items = document.querySelectorAll(".item");
items.forEach(el => console.log(el.textContent));

// Other selectors
document.getElementsByClassName("note");
document.getElementsByTagName("p");

// Parent / children / siblings
el.parentElement;
el.children;
el.nextElementSibling;
el.previousElementSibling;
```

`querySelector` and `querySelectorAll` use CSS selectors and are the most flexible.

Changing content and attributes

```
const el = document.querySelector("#title");

el.textContent = "New title";      // safe text
el.innerHTML   = "<em>HTML</em>";  // parses HTML

el.setAttribute("data-id", "42");
el.getAttribute("data-id");        // "42"
el.removeAttribute("data-id");

el.classList.add("active");
el.classList.remove("hidden");
el.classList.toggle("selected");
el.classList.contains("active");   // true

el.dataset.userId = "42";          // data-user-id="42"
```

Prefer `textContent` for user input. `innerHTML` can introduce XSS if the content is untrusted.

Creating and inserting elements

```
const list = document.querySelector("#list");

const item = document.createElement("li");
item.textContent = "New item";
item.classList.add("item");

list.appendChild(item);            // add at end
list.prepend(item);                // add at start
list.insertBefore(item, list.firstChild);
list.replaceChild(newItem, oldItem);

item.remove();                     // remove from DOM
list.innerHTML = "";               // clear all children
```

Build content with `createElement` and `textContent` to avoid unsafe HTML injection.

Handling events

```
const btn = document.querySelector("#save");

btn.addEventListener("click", (event) => {
  console.log("Clicked", event.target);
});

document.addEventListener("keydown", (e) => {
  if (e.key === "Escape") {
    console.log("Escape pressed");
  }
});

form.addEventListener("submit", (e) => {
  e.preventDefault();   // stop page reload
  // handle form data
});
```

Common events: `click`, `input`, `change`, `submit`, `keydown`, `mouseover`, `load`.

Event delegation

```
// Instead of adding a listener to every item:
document.querySelector("#list")
  .addEventListener("click", (e) => {
    if (e.target.matches("li.item")) {
      console.log("Clicked:", e.target.textContent);
    }
  });
```

Why it works

Events bubble up from the target to ancestors. One listener on a parent can handle clicks on many children — including ones added later.

Bubbling, capturing, and stopPropagation

```
parent.addEventListener("click", () => console.log("parent"));
child.addEventListener("click", (e) => {
  e.stopPropagation(); // prevent bubbling
  console.log("child");
});

// Capture phase (rarely used)
parent.addEventListener("click", handler, true);
```

- **Capturing:** event travels from root down to target.
- **Bubbling:** event travels from target back up.
- `stopPropagation` stops further propagation; `preventDefault` cancels default behavior.

Working with CSS from JavaScript

```
// Inline styles
el.style.color = "white";
el.style.backgroundColor = "indigo";
el.style.padding = "8px";
el.style.setProperty("--brand", "#00ACC1");

// Classes (preferred)
el.classList.add("card--active");
el.classList.remove("card--hidden");

// Reading computed styles
getComputedStyle(el).color;
```

Best practice

Prefer toggling classes over setting many inline styles. Keep visual rules in CSS, and let JavaScript decide *which* class to apply.

Reading form data

```
const form = document.querySelector("#signup");

form.addEventListener("submit", (e) => {
  e.preventDefault();

  const data = new FormData(form);
  const name = data.get("name");
  const email = data.get("email");

  // Or collect all fields
  const values = Object.fromEntries(data.entries());
  console.log(values);
});
```

`FormData` is the easiest way to read form fields by their `name` attribute. Validate before sending.

Part 7 — Object-Oriented

JavaScript

Classes

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  greet() {
    return `Hi, I am ${this.name}`;
  }

  get info() {
    return `${this.name} (${this.age})`;
  }

  static species() {
    return "Homo sapiens";
  }
}

const ada = new Person("Ada", 36);
ada.greet();
Person.species();
```

Inheritance and private fields

```
class Animal {
  constructor(name) { this.name = name; }
  speak() { return `${this.name} makes a sound`; }
}

class Dog extends Animal {
  constructor(name, breed) {
    super(name);
    this.breed = breed;
  }
  speak() { return `${this.name} barks`; }
}

// Private fields
class Counter {
  #count = 0;
  increment() { this.#count++; }
  get value() { return this.#count; }
}
```

Use `extends` for inheritance and `super()` to call the parent constructor. `#field` creates truly private fields.

Prototypes (the foundation under classes)

```
function Person(name) {
  this.name = name;
}
Person.prototype.greet = function () {
  return `Hi, ${this.name}`;
};

const p = new Person("Ada");
p.greet(); // "Hi, Ada"

// Every object has a prototype chain
Object.getPrototypeOf(p) === Person.prototype; // true
```

ES6 `class` is syntactic sugar over prototypes. Understanding prototypes helps when debugging and reading older code.

Part 8 — Asynchronous JavaScript

Synchronous vs asynchronous

Synchronous

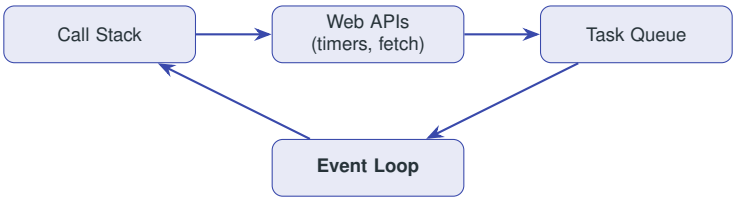
- Code runs line by line.
- Each operation waits for the previous one.
- Simple to reason about.

Asynchronous

- Long operations do not block the thread.
- Callbacks, Promises, `async/await`.
- Essential for network and file I/O.

JavaScript runs on a single thread but uses an event loop to handle asynchronous tasks without blocking the UI.

The Event Loop (simplified)



When the stack is empty, the event loop moves the next queued task onto the stack. Promises use a separate microtask queue that runs before the task queue.

setTimeout and setInterval

```
// Run once after 1 second
setTimeout(() => {
  console.log("After 1s");
}, 1000);

// Run every 2 seconds
const id = setInterval(() => {
  console.log("Tick");
}, 2000);

// Stop it
clearInterval(id);

// Cancel a timeout
const t = setTimeout(() => {}, 500);
clearTimeout(t);
```

Timers are not exact. Delay is a minimum, not a guarantee.

Callbacks and callback hell

```
// Callback style
getUser(1, (user) => {
  getOrders(user.id, (orders) => {
    getOrderDetails(orders[0].id, (details) => {
      console.log(details);
      // deeply nested...
    });
  });
});
```

The problem

Nested callbacks become hard to read, hard to debug, and hard to handle errors in. Promises and `async/await` solve this.

Promises

```
const p = new Promise((resolve, reject) => {
  const ok = true;
  if (ok) resolve("done");
  else reject(new Error("failed"));
});

p.then(value => console.log(value))
 .catch(err => console.error(err))
 .finally(() => console.log("finished"));
```

- A Promise has three states: pending, fulfilled, rejected.
- `.then`, `.catch`, `.finally` chain.
- `Promise.all`, `Promise.race`, `Promise.allSettled` combine promises.

async / await

```
async function loadUser(id) {
  try {
    const response = await fetch(`/api/users/${id}`);
    if (!response.ok) throw new Error("HTTP " + response.status);
    const user = await response.json();
    return user;
  } catch (err) {
    console.error("Failed:", err);
  }
}
```

`async` makes a function return a Promise. `await` pauses until the Promise settles. Always wrap in `try/catch`.

Running promises in parallel

```
// Sequential (slower)
const a = await fetch("/a");
const b = await fetch("/b");

// Parallel (faster)
const [ra, rb] = await Promise.all([
  fetch("/a"),
  fetch("/b"),
]);

// Settled (does not throw on failure)
const results = await Promise.allSettled([
  fetch("/a"),
  fetch("/b"),
]);
```

`Promise.all` fails fast if any promise rejects. `allSettled` waits for all and reports each result.

Part 9 — APIs & Storage

Fetching data with fetch

```
async function loadUsers() {
  try {
    const response = await fetch(
      "https://jsonplaceholder.typicode.com/users"
    );
    if (!response.ok) {
      throw new Error(`HTTP ${response.status}`);
    }
    const users = await response.json();
    console.log(users);
  } catch (err) {
    console.error("Request failed:", err);
  }
}

loadUsers();
```

`fetch` returns a Promise. Always check `response.ok` and handle errors with `try/catch`.

Sending data with fetch (POST)

```
const newUser = { name: "Ada", age: 36 };

const response = await fetch("/api/users", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
  },
  body: JSON.stringify(newUser),
});

if (response.ok) {
  const created = await response.json();
  console.log("Created:", created);
}
```

Set `Content-Type: application/json` and `JSON.stringify` the body. The server must allow the request (CORS).

Fetching data with Axios

```
// Include via CDN or npm:
// <script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>

async function loadUsers() {
  try {
    const { data } = await axios.get(
      "https://jsonplaceholder.typicode.com/users"
    );
    console.log(data);
  } catch (err) {
    console.error(err.response?.status, err.message);
  }
}

// POST
await axios.post("/api/users", { name: "Ada" });
```

Axios automatically parses JSON, throws on HTTP errors, and supports interceptors. `fetch` requires manual `response.json()`.

CORS and common API errors

- **CORS:** browsers block cross-origin requests unless the server sends `Access-Control-Allow-Origin`.
- **401/403:** missing or invalid credentials.
- **404:** wrong URL.
- **429:** rate limited.
- **500:** server error.

Debugging tips

Open the Network tab in DevTools, inspect the request and response headers, and read the error message carefully.

Cancelling requests with AbortController

```
const controller = new AbortController();

fetch("/api/data", { signal: controller.signal })
  .then(r => r.json())
  .catch(err => {
    if (err.name === "AbortError") {
      console.log("Request cancelled");
    }
  });

// Cancel after 2 seconds
setTimeout(() => controller.abort(), 2000);
```

Useful for search-as-you-type, navigation away, and timeouts.

localStorage and sessionStorage

```
// Store (strings only)
localStorage.setItem("theme", "dark");
localStorage.setItem("user", JSON.stringify({ name: "Ada" }));

// Read
const theme = localStorage.getItem("theme");      // "dark"
const user  = JSON.parse(localStorage.getItem("user"));

// Update / delete
localStorage.setItem("theme", "light");
localStorage.removeItem("theme");
localStorage.clear();

// Size limit ~5MB. Synchronous. Persists across sessions.
```

`sessionStorage` behaves the same but is cleared when the tab closes. For large data, use IndexedDB.

Cookies (briefly)

```
// Write (very simple)
document.cookie = "theme=dark; max-age=86400; path=/";

// Read (parsing required)
console.log(document.cookie); // "theme=dark; other=..."

// Delete
document.cookie = "theme=; max-age=0; path=/";
```

- Cookies are sent with every HTTP request to the matching domain.
- Use `HttpOnly` and `Secure` cookies on the server for auth tokens.
- Prefer `localStorage` for client-only data; use cookies for session/auth.

Storage comparison

Feature	localStorage	sessionStorage	Cookie
Lifetime	Persistent	Tab session	Configurable
Size	5 MB	5 MB	4 KB
Sent to server	No	No	Yes
API	Simple	Simple	Manual
Use case	Preferences	Wizard state	Auth/session

Part 10 — Patterns & Practice

ES Modules

math.js

```
export const PI = 3.14;
export function add(a, b) { return a + b; }
export default function mul(a, b) {
  return a * b;
}
```

app.js

```
import mul, { PI, add } from "./math.js";
console.log(add(2, 3), mul(2, 3), PI);
```

Requirements

Use `<script type="module">` in the browser, or a bundler/Node.js. Modules have their own scope and are loaded once.

Debounce and throttle

Debounce

```
function debounce(fn, delay) {
  let t;
  return (...args) => {
    clearTimeout(t);
    t = setTimeout(() => fn(...args), delay);
  };
}
```

Run after the user stops.

Throttle

```
function throttle(fn, limit) {
  let last = 0;
  return (...args) => {
    const now = Date.now();
    if (now - last >= limit) {
      last = now;
      fn(...args);
    }
  };
}
```

Run at most once per interval.

Strict mode

```
"use strict";

// Without strict mode:
x = 10; // creates a global (bad)

// With strict mode:
// ReferenceError: x is not defined

// Other effects:
// - this is undefined in plain function calls
// - duplicate parameter names throw
// - silent failures become errors
```

ES Modules and classes are automatically strict. Prefer modules over global `"use strict"`.

Debugging in DevTools

- **Console:** log values, run expressions.
- **Sources:** set breakpoints, step through code.
- **Network:** inspect requests, responses, timing.
- **Elements:** inspect and edit the DOM.
- **Application:** inspect localStorage, cookies, IndexedDB.
- `debugger;` statement pauses execution when DevTools is open.

Tip

Read the first line of an error message, then the stack trace. The bug is usually a few lines above where the error appears.

Common beginner mistakes

- Using `var` everywhere instead of `const`/`let`.
- Comparing with == instead of ===.
- Forgetting `break` in `switch`.
- Mutating arrays unexpectedly with `sort`/`splice`.
- Losing `this` when passing methods as callbacks.
- Using `innerHTML` with untrusted data (XSS).
- Forgetting `await` or `try/catch` around async code.
- Blocking the UI with heavy synchronous loops.
- Storing complex objects in localStorage without `JSON.stringify`.

Practice lab

Build a small "Task Manager" app using only vanilla JavaScript.

1. Create an HTML form with an input and a submit button.
2. On submit, add the task to an array and render it with `createElement`.
3. Use `localStorage` to persist tasks across reloads.
4. Add a delete button per task using event delegation.
5. Add a filter (All / Done / Pending) using `filter` and `map`.
6. Style the app with CSS classes toggled from JavaScript.
7. Fetch a list of sample tasks from `jsonplaceholder.typicode.com/todos` with `fetch` or Axios.

Success criteria

The app works offline, is accessible from the keyboard, and separates HTML, CSS, and JS into three files.

Recap

You learned

- History, syntax, and inclusion methods.
- Variables, types, operators, coercion.
- Functions, closures, `this`, `call/apply/bind`.
- Arrays, strings, objects, Map/Set.
- Control flow and array iteration.
- Math, Date, JSON, Intl, Regex.
- DOM, events, CSS from JS.
- OOP, prototypes, private fields.
- Async, Event Loop, Promises, `async/await`.
- fetch, Axios, CORS, AbortController.
- localStorage, sessionStorage, cookies.
- Modules, debounce/throttle, strict mode.

Suggested next steps

- Build 2–3 small projects.
- Learn Node.js and Express.
- Learn React, Vue, or Svelte.
- Learn Jest or Vitest.
- Learn TypeScript.
- Learn Vite or esbuild.
- Learn REST, SQL, and auth.

Learn the language.

Build small things.

Then build bigger things.

JavaScript Foundations — end of course